

Día 5: Método de Lee-Carter: modelación y proyección

Basado en las notas de Ugofilippo Basellini

José Manuel Aburto y Ana Cristina Gómez Ugarte Valerio

2026

Tabla de contenidos

Carga de datos	2
Preprocesamiento de datos	2
Análisis exploratorio	3
Mortalidad observada por edad	3
Esperanza de vida observada	3
Estimación del Modelo de Lee-Carter	4
Paso 1: Estimación de α (componente de edad promedio)	4
Paso 2: Estimación de β y κ	5
Estimación de la matriz de residuos	5
Descomposición en Valores Singulares (SVD)	6
Incluir las restricciones	7
Ajustar kappa para mantener el número de defunciones en cada año	8
Valores ajustados de mortalidad	9
Paso 3: Proyección de Mortalidad	11
Modelo de Series de Tiempo para κ	11
Simulación de Escenarios	12
Cálculo de intervalos de confianza	13
Proyección de tasas de mortalidad y esperanza de vida	15

Este documento implementa el método de Lee-Carter para modelar y proyectar la mortalidad utilizando datos de Inglaterra y Gales. El archivo `lee-carter-data.RData` contiene el número de muertes observadas (Y), la población expuesta al riesgo (E) y las tasas de mortalidad observadas en escala logarítmica (LMX). Los datos proceden de la Human Mortality Database.

Carga de datos

```
# Cargar las funciones de tabla de vida
source("funciones-tabla-vida.R")

# Cargar datos de Inglaterra y Gales
load("lee-carter-data.RData")
```

Preprocesamiento de datos

Tenemos datos observados de 1960 a 2018, pero vamos a seleccionar 1970 como el año inicial. Filtramos los datos a partir de este año y definimos nuestro horizonte de proyección, queremos proyectar hasta 2040.

```
# Configuración inicial
initial.year <- 1970
initial.year.ind <- initial.year - 1959 # Columna correspondiente a 1970
last.year.ind <- ncol(LMX)

# Seleccionar datos a partir de 1970
LMX <- LMX[, initial.year.ind:last.year.ind]
E <- E[, initial.year.ind:last.year.ind]
Y <- Y[, initial.year.ind:last.year.ind]

# Filtrar vectores de tiempo
t <- t[t >= initial.year]
t1 <- t1[t1 >= initial.year]

# Definimos las dimensiones
m <- length(a)      # Número de edades
n <- length(t)       # Número de años observados
n1 <- length(t1)     # Número de años para proyección
nF <- length(tF)     # Número de años futuros a proyectar
```

Análisis exploratorio

Mortalidad observada por edad

```
par(mar = c(3, 3, 1, 1))  
matplot(a, LMX, type = "l", col = viridis(n1), lty = 1,  
        xlab = "Edad", ylab = "Tasa de mortalidad (log)")  
mtext("Edad",side=1,line=1.5)  
mtext("Tasa de mortalidad (log)",side=2,line=2)
```

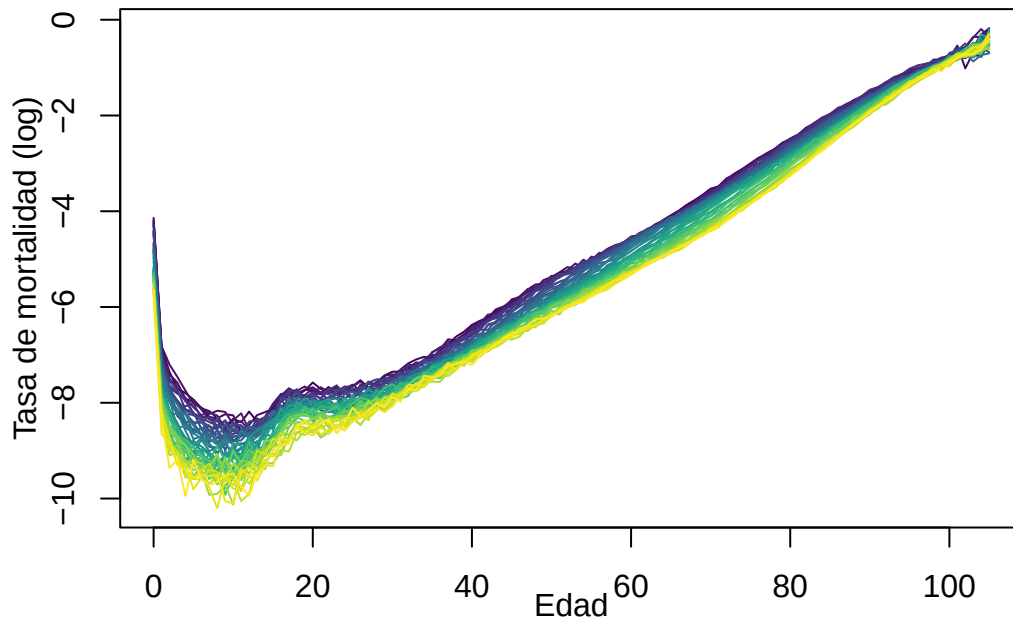


Figura 1: Tasas de mortalidad observadas por edad (escala logarítmica).

```
rany <- range(LMX, finite = TRUE)
```

Esperanza de vida observada

```
# Calcular esperanza de vida observada  
e0obs <- apply(exp(LMX), 2, e0.mx, x = a, sex = "F")  
  
# Graficar
```

```
plot(t1, e0obs, pch = 16, col = "blue",
     xlab = "Año", ylab = "e0", main = "Esperanza de vida al nacimiento")
```

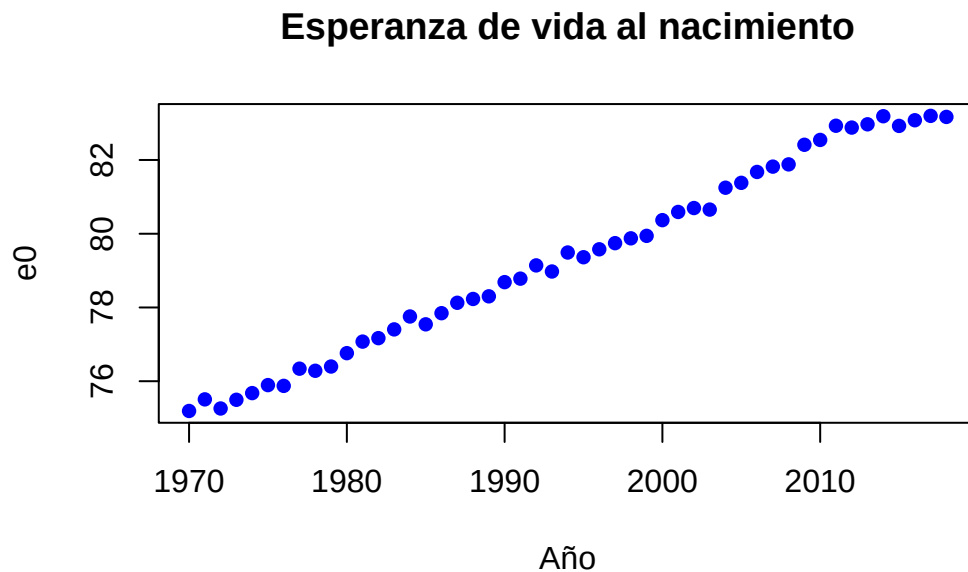


Figura 2: Esperanza de vida observada

Estimación del Modelo de Lee-Carter

Paso 1: Estimación de α (componente de edad promedio)

Para estimar el modelo de Lee-Carter, primero tenemos que estimar α . Recordemos que es la tasa promedio de mortalidad para cada edad. Así obtenemos un patrón promedio de mortalidad por edad.

```
# Estimamos alpha
Alpha <- apply(LMX,1,mean)

## Gráfica
matplot(a,LMX,t="l",lty=1,lwd=0.8,col="grey70",xlab="Edad",ylab="Tasa de mortalidad (log)")
lines(a,Alpha,col="darkgreen",lwd=2)
```

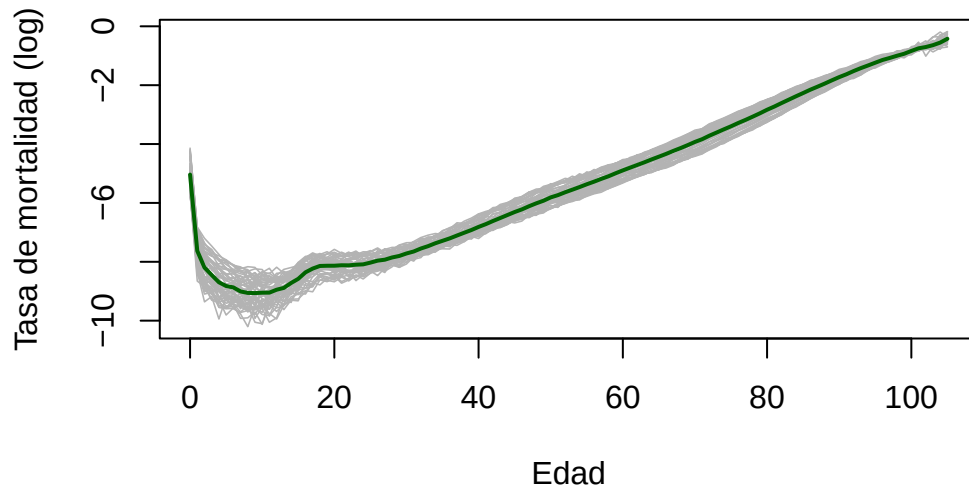


Figura 3: Componente de edad promedio (α) comparado con tasas de mortalidad anuales observadas.

Paso 2: Estimación de β y κ

El segundo paso consiste en estimar los residuos y realizar la descomposición en valores singulares (SVD) de la matriz.

Estimación de la matriz de residuos

```
# Estimamos la matriz de residuos (mortalidad "centrada")
LMXres <- LMX - Alpha

# Gráfica
image.plot(t1,a,t(LMXres),axes="F", xlab = "Año", ylab = "Edad")
axis(1);axis(2,las=2);box()
```

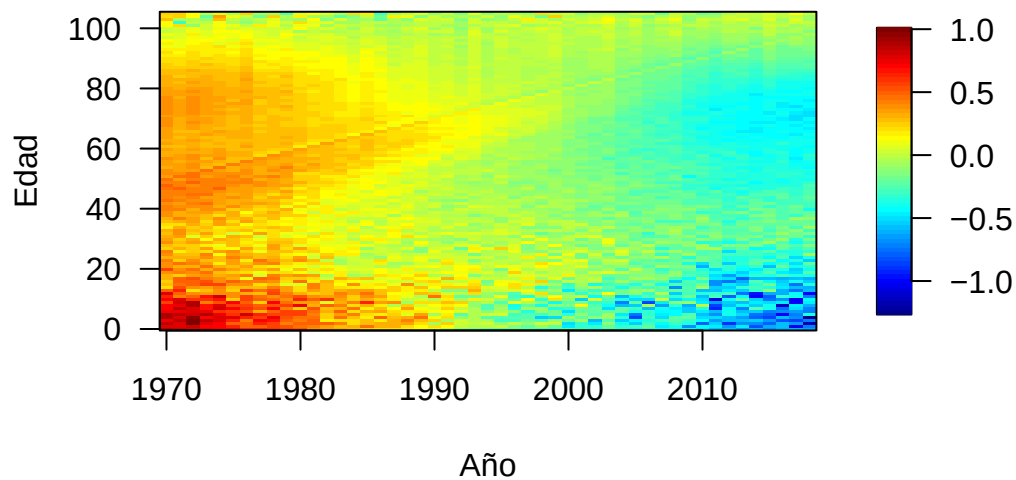


Figura 4: Matriz de residuos (mortalidad centrada)

Descomposición en Valores Singulares (SVD)

Realizamos la descomposición en valores singulares (SVD) de la matriz de residuos y extraemos los primeros vectores singulares izquierdo y derecho.

```
# Realizar descomposición
LCsvd <- svd(LMXres, nu = 1, nv = 1)

# Extraer vectores singulares
Beta <- c(LCsvd$u)
Kappa1 <- c(LCsvd$v)

# Gráfica
par(mfrow = c(1, 2))
plot(a, Beta, main = expression(beta), xlab = "Edad", ylab = "")
plot(t1, Kappa1, main = expression(kappa[1]), xlab = "Año", ylab = "")
```

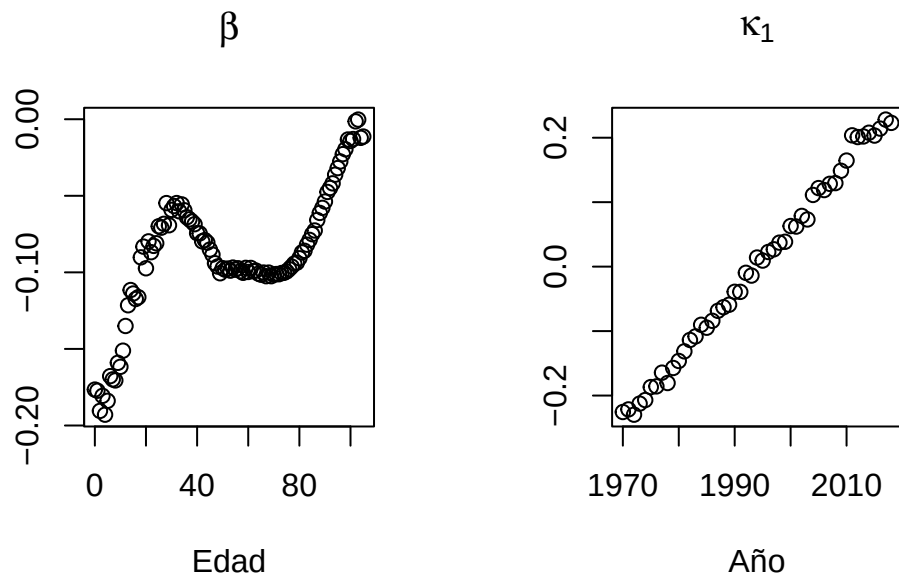


Figura 5: Vectores singulares

Incluir las restricciones

Recordemos que tenemos dos restricciones: $\sum_x \beta_x = 1$ y $\sum_t \kappa_t = 0$. Vamos a implementarlas.

```
# Restricción 1: Normalizar beta
sum.Beta <- sum(Beta)
Beta <- Beta / sum.Beta
cat("Suma de beta después de normalizar:", sum(Beta), "\n")
```

Suma de beta después de normalizar: 1

```
# Restricción 2: Escalar kappa
Kappa1 <- LCsvd$d[1] * Kappa1 * sum.Beta
cat("Suma de kappa1 después de escalar:", sum(Kappa1), "\n")
```

Suma de kappa1 después de escalar: -3.552714e-14

```
# Graficamos los componentes ajustados
par(mfrow = c(1, 2))
plot(a, Beta, main = expression(beta), xlab = "Edad", ylab = "")
plot(t1, Kappa1, main = expression(kappa[1]), xlab = "Año", ylab = "")
```

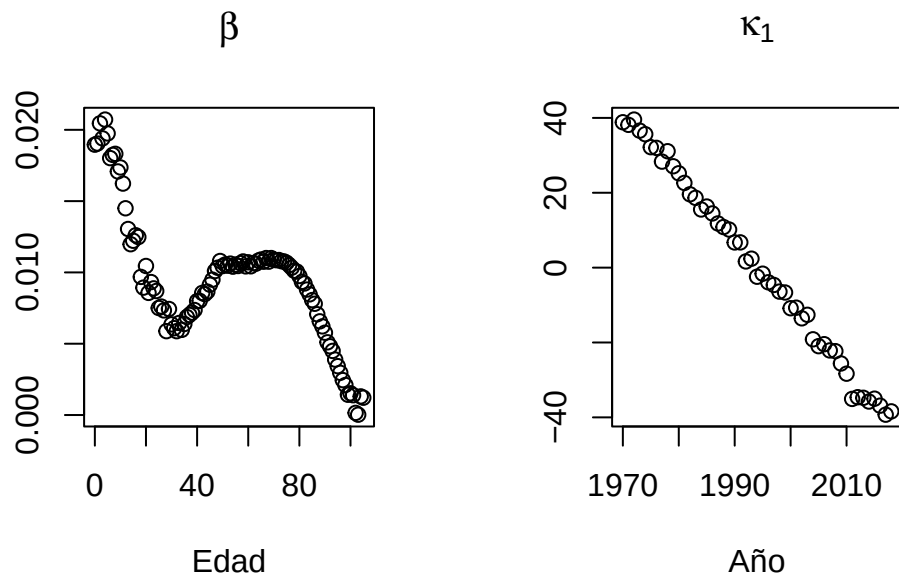


Figura 6: Beta y kappa ajustadas

Ejercicio: ¿Cómo interpretarías cada uno de estos parámetros?

Ajustar kappa para mantener el número de defunciones en cada año

```
# Función de optimización
koptim <- function(par, alpha, beta, sum.dx, Exp) {
  kappa <- par[1]
  lmx.lc <- alpha + beta * kappa
  z.lc <- exp(lmx.lc) * Exp
  sum.z.lc <- sum(z.lc)
  diff.lc <- abs(sum.dx - sum.z.lc)
  return(diff.lc)
}

# Estimar kappa para cada año
Kappa <- numeric(n1)

for (i in 1:n1) {
  KappaSecStep <- optimize(f = koptim,
                           interval = c(-100, 100),
                           alpha = Alpha,
                           beta = Beta,
```

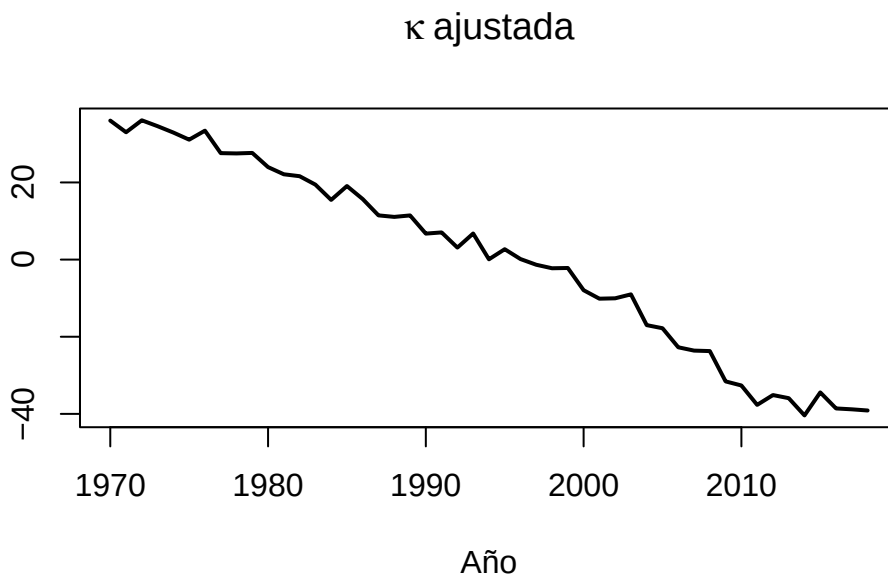


```

        sum.dx = sum(Y[, i]),
        Exp = E[, i])
    Kappa[i] <- KappaSecStep$minimum
}

# Gráfica
plot(t1, Kappa, type = "l", lwd = 2,
     main = expression(kappa ~ ajustada),
     xlab = "Año", ylab = "")

```



Valores ajustados de mortalidad

Recordemos que $\ln(m_{x,t}) = \alpha_x + \beta_x \kappa_t$. Una vez estimados los parámetros, podemos obtener los valores ajustados de mortalidad.

```

# Calcular ETA (tasas de mortalidad en escala logarítmica) ajustada
Ones <- matrix(1, n1)
ETAlc <- Alpha %*% t(Ones) + Beta %*% t(Kappa)

# Graficar mortalidad ajustada
matplot(a, ETAlc, type = "l", lty = 1, lwd = 0.8,
       col = viridis(n1, option = "A"),
       xlab = "Edad", ylab = "Tasa de mortalidad (log)")

```

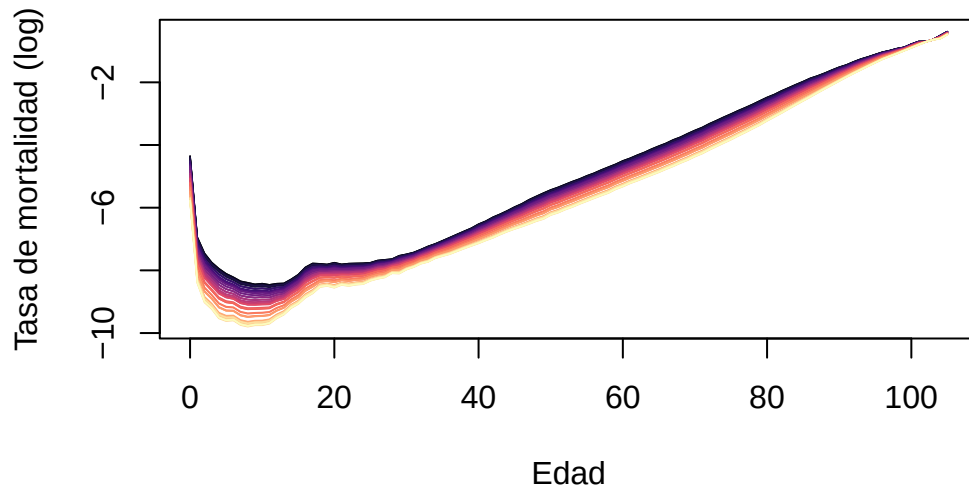


Figura 7: Mortalidad ajustada por año.

```
# Esperanza de vida ajustada
e0lc <- apply(exp(ETAlc), 2, e0.mx, x = a, sex = "F")

# Comparar esperanza de vida observada vs ajustada
plot(t1, e0obs, pch = 1, col = "blue", lwd = 2,
     xlab = "", ylab = "", main = "Esperanza de vida: observada vs ajustada")
mtext("Años", side = 1, line = 1.5)
mtext(expression(e[x]), side = 2, line = 2)
points(t1, e0lc, pch = 4, col = "orange", lwd = 2)
legend("topleft",
     legend = c("Observada", "Ajustada"),
     col = c("blue", "orange"), pch = c(1, 4), lwd = 2)
```

Esperanza de vida: observada vs ajustada

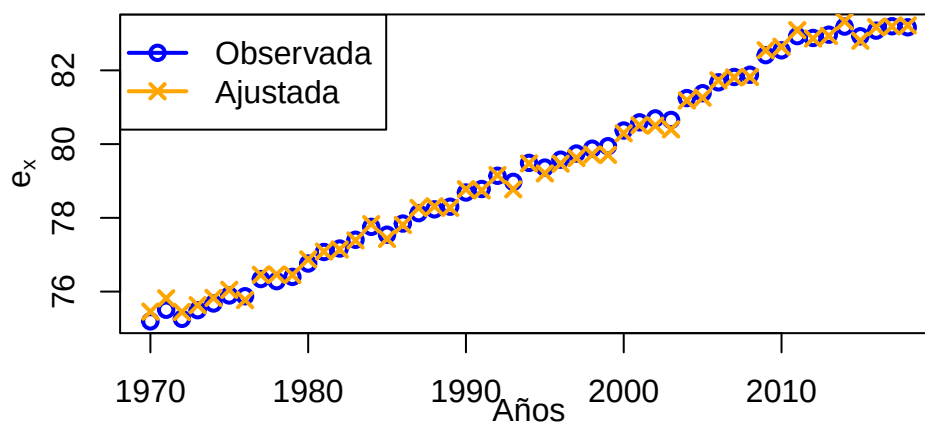


Figura 8: Comparación de esperanza de vida observada y ajustada por año.

Paso 3: Proyección de Mortalidad

Modelo de Series de Tiempo para κ

```
# Crear objeto ts para kappa
Kts <- ts(c(Kappa), start = t1[1])

# Ajustar modelo ARIMA(0,1,0) con deriva
modK <- Arima(Kts, order = c(0, 1, 0), include.drift = TRUE)

# Proyectar kappa
predK <- forecast(modK, h = nF)

# Graficar proyección
plot(predK, main = "Proyección de kappa")
```

Proyección de kappa

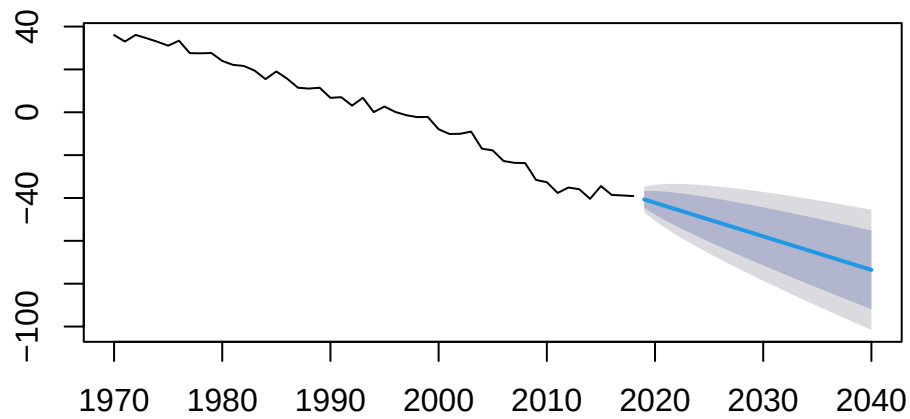


Figura 9: Proyección de kappa usando ARIMA(0,1,0) con deriva.

```
# Otra forma de visualizar los resultados
plot(t1,Kappa,t="l",lwd=2,ylim=range(Kappa,predK$lower),xlim=range(t))
lines(tF,predK$mean,col=2,lwd=2)
matlines(tF,predK$upper,col=c(3,4),lwd=2)
matlines(tF,predK$lower,col=c(3,4),lwd=2)
```

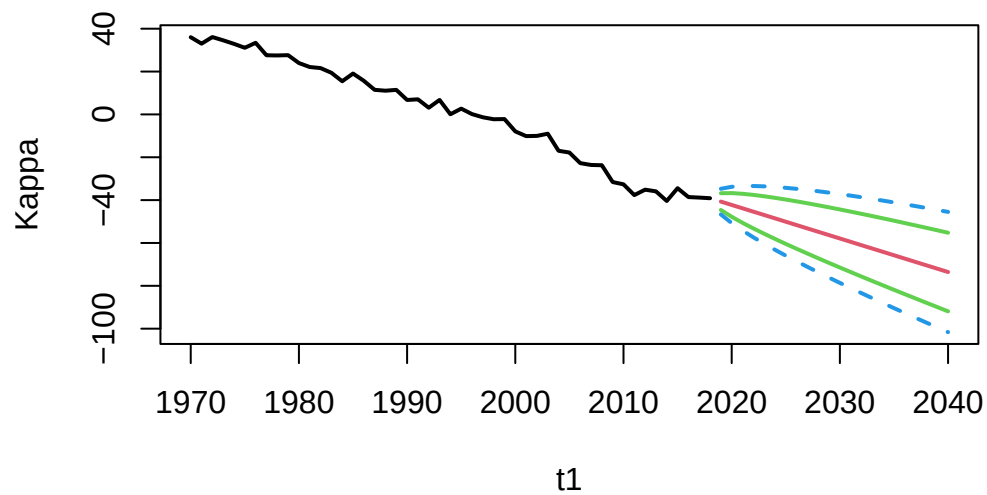


Figura 10: Proyección de kappa usando ARIMA(0,1,0) con deriva.

Simulación de Escenarios

Utilicemos el modelo para generar 200 simulaciones de valores futuros para kappa.

```

# Definir número de simulaciones y configurar matriz para guardar resultados
nS <- 200
SIMk <- matrix(NA, nrow = nF, ncol = nS)
set.seed(2026)

# Generar simulaciones
for (i in 1:nS) {
  kappa.sim <- simulate(modK, nsim = nF,
                        future = TRUE, bootstrap = TRUE)
  SIMk[, i] <- kappa.sim
}

# Graficar simulaciones
plot(t1, Kappa, ylim = range(Kappa, SIMk), xlim = range(t),
     main = "Simulaciones de kappa", xlab = "Año", ylab = expression(kappa))
matlines(tF, SIMk, col = "grey70", lwd = 0.5)
lines(tF, predK$mean, col = "red", lwd = 2)

```

Simulaciones de kappa

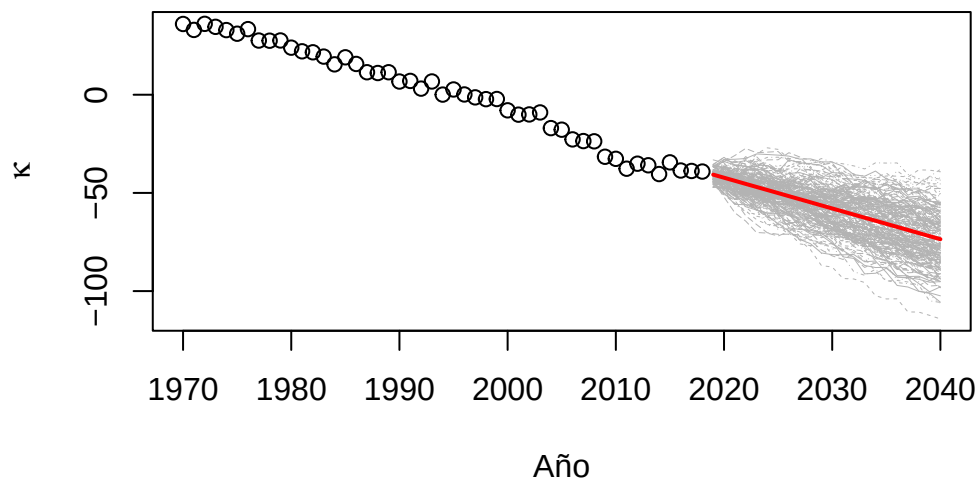


Figura 11: Simulaciones para kappa.

Cálculo de intervalos de confianza

Podemos calcular los intervalos de predicción de dos formas: analíticamente, utilizando los resultados del modelo ARIMA, o a partir de las simulaciones. Vamos a comparar ambos métodos.

```

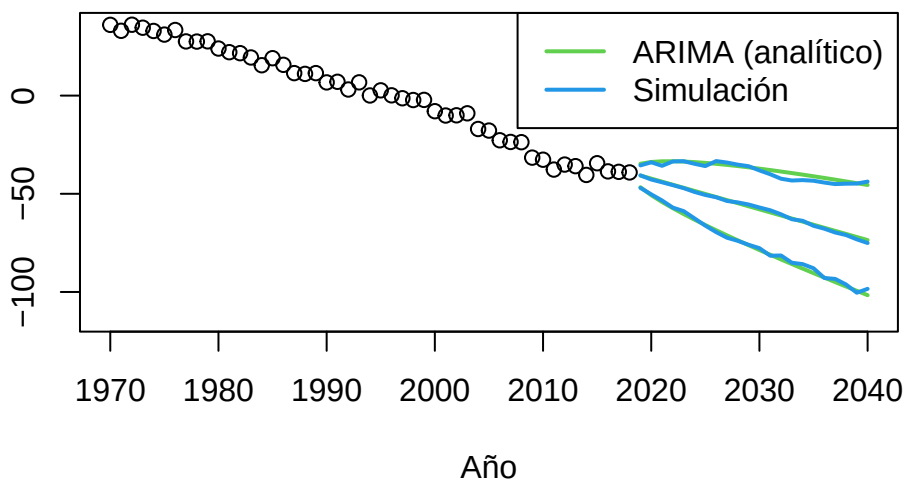
# Nivel de confianza
lev <- 95
lev.p <- lev / 100

# Calcular medianas y límites de los intervalos
Kappa.low <- apply(SIMk, 1, quantile, prob = (1 - lev.p) / 2)
Kappa.upp <- apply(SIMk, 1, quantile, prob = 1 - (1 - lev.p) / 2)
Kappa.med <- apply(SIMk, 1, median)

# Comparar intervalos
plot(t1, Kappa, ylim = range(Kappa, SIMk), xlim = range(t),
     main = "Intervalos de confianza para kappa", xlab = "Año", ylab = "")
lines(tF, predK$lower[, 2], col = 3, lwd = 2) # Analítico
lines(tF, predK$upper[, 2], col = 3, lwd = 2)
lines(tF, predK$mean, col = 3, lwd = 2)
lines(tF, Kappa.low, col = 4, lwd = 2)        # Simulación
lines(tF, Kappa.upp, col = 4, lwd = 2)
lines(tF, Kappa.med, col = 4, lwd = 2)
legend("topright",
      legend = c("ARIMA (analítico)", "Simulación"),
      col = c(3, 4), lwd = 2)

```

Intervalos de confianza para kappa



Proyección de tasas de mortalidad y esperanza de vida

Por último utilizaremos la proyección de kappa para proyectar las tasas de mortalidad por edad y la esperanza de vida. Las simulaciones permiten estimar la esperanza de vida (o cualquier otro indicador de mortalidad) en cada trayectoria y obtener después la mediana y los intervalos de predicción del conjunto de valores estimados.

```
# Crear objetos para almacenar los resultados
ETAlc.fore <- array(NA, dim = c(m, nF, nS))
e0lc.fore <- matrix(NA, nF, nS)

# Para cada simulación, calcular las tasas de mortalidad proyectadas y la esperanza de vida
for (i in 1:nS) {
  Ones <- matrix(1, nF)
  ETAlc.fore[, , i] <- Alpha %*% t(Ones) + Beta %*% t(SIMk[, i])
  e0lc.fore[, i] <- apply(exp(ETAlc.fore[, , i]), 2, e0.mx, x = a, sex = "F")
}

# Calcular medianas e intervalos de confianza para e0
e0lc.fore.low <- apply(e0lc.fore, 1, quantile, prob = (1 - lev.p) / 2)
e0lc.fore.upp <- apply(e0lc.fore, 1, quantile, prob = 1 - (1 - lev.p) / 2)
e0lc.fore.med <- apply(e0lc.fore, 1, median)

# Calcular medianas e intervalos para las tasas de mortalidad estimadas (ETA)
ETAlc.fore.low <- ETAlc.fore.upp <- ETAlc.fore.med <- matrix(NA, m, nF)

for (i in 1:m) {
  ETAlc.fore.low[i, ] <- apply(ETAlc.fore[i, , ], 1, quantile,
                              prob = (1 - lev.p) / 2)
  ETAlc.fore.upp[i, ] <- apply(ETAlc.fore[i, , ], 1, quantile,
                              prob = 1 - (1 - lev.p) / 2)
  ETAlc.fore.med[i, ] <- apply(ETAlc.fore[i, , ], 1, median)
}

# Gráfica de la tasa de mortalidad para una edad específica
my.age <- 0
ylim <- range(LMX[which(a == my.age), ],
              ETAlc.fore.low[which(a == my.age), ])

plot(t1, LMX[which(a == my.age), ], type = "n",
     xlab = "Año", ylab = "Tasa de mortalidad (log)", ylim = ylim,
     xlim = range(t), axes = FALSE)
axis(1); axis(2, las = 2)
```

```

grid(); box()
title(paste("Edad", my.age))
points(t1, LMX[which(a == my.age), ], pch = 16, col = "grey40", cex = 0.75)
lines(t1, ETAlc[which(a == my.age), ], col = "blue", lwd = 2)
matlines(tF, ETAlc.fore[which(a == my.age), , ], col = "grey70")
lines(tF, ETAlc.fore.med[which(a == my.age), ], col = "darkblue", lwd = 2)
lines(tF, ETAlc.fore.low[which(a == my.age), ], col = "darkblue", lwd = 2, lty = 2)
lines(tF, ETAlc.fore.upp[which(a == my.age), ], col = "darkblue", lwd = 2, lty = 2)

```

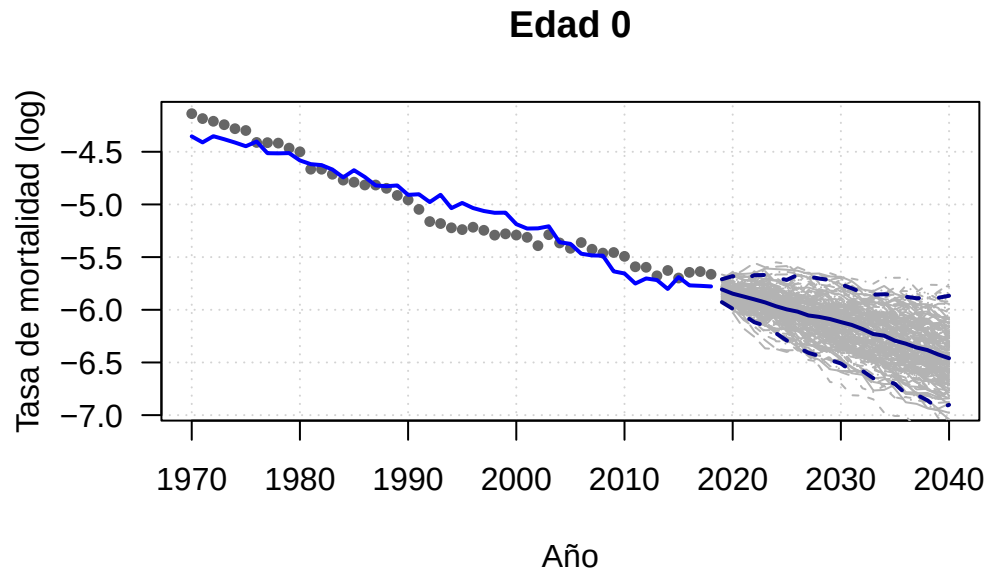


Figura 12: Proyección de mortalidad para edad 0.

```

# Gráfica de la esperanza de vida
ylim <- range(e0obs, e0lc, e0lc.fore)

plot(t1, e0obs, type = "n", xlab = "Año", ylab = "Esperanza de vida al nacimiento",
      ylim = ylim, xlim = range(t), axes = FALSE)
axis(1); axis(2, las = 2); grid(); box()
points(t1, e0obs, pch = 16, col = "grey40", cex = 0.75)
lines(t1, e0lc, col = "blue", lwd = 2)
matlines(tF, e0lc.fore, col = "grey70")
lines(tF, e0lc.fore.med, col = "darkblue", lwd = 2)
lines(tF, e0lc.fore.upp, col = "darkblue", lwd = 2, lty = 2)
lines(tF, e0lc.fore.low, col = "darkblue", lwd = 2, lty = 2)

```

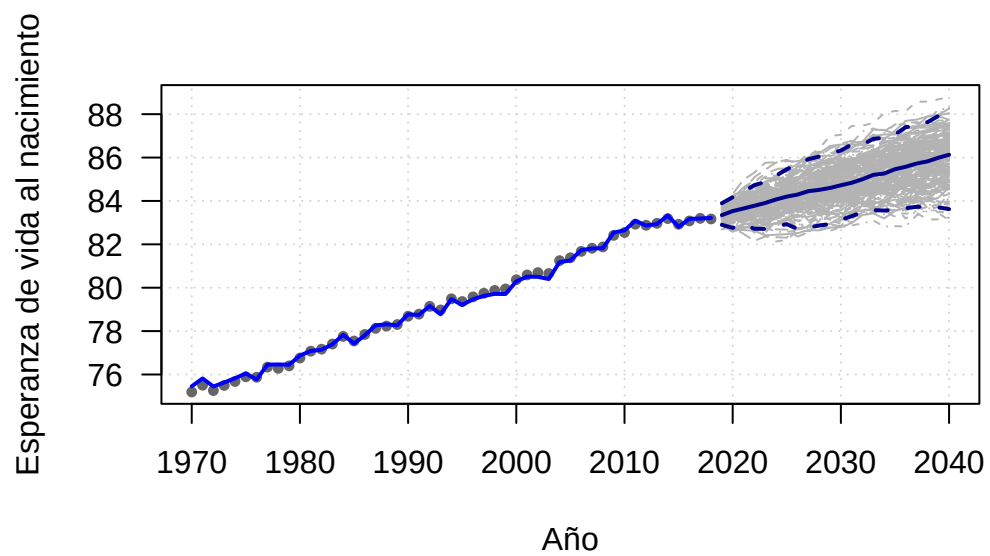



Figura 13: Proyección de esperanza de vida al nacimiento.